

Code-Layout, Readability and Reusability

Date	5 July 2026
Team ID	SWTID-2026-4860
Project Name	EduGenie Google Gemini Powered Learning Assistant
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	Standard PEP 8 formatting applied using 4 spaces per indentation level.
2	Proper File Structure	Files and folders are logically organized	Yes	Separated into modular backend (main.py), static styles (style.css), and templates (index.html).
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Descriptive naming conventions like GEMINI_KEY, lamini_pipeline, and active_tab are used.
4	Function / Method Names	Functions are descriptively named	Yes	Execution route and helper handlers are clearly labeled (e.g., call_gemini, handle_summarize).
5	Code Comments	Inline and block comments explain logic	Yes	Clear docstrings and structural block comments explain the AI initialization steps and prompt processing routes.
6	Modular Design	Code is split into reusable functions/modules	Yes	Both cloud and local LLM pipelines are encapsulated into completely independent helper functions.
7	No Redundant Code	Duplicate or unused code is removed	Yes	Unused imports (like the legacy generativeai and unused pydantic schemas) were thoroughly cleaned.
8	Error Handling	Exceptions and errors are handled gracefully	Yes	try-except blocks to catch API/Inference errors and added a strict validator for environment keys.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	Gemini Cloud Inference Wrapper(call_gemini)	Python / google-genai	Called inside /qa, /explain, /quiz, and /recommend endpoints to route prompts.	High

2	Local Summarization Pipeline (call_lamini)	Python / HuggingFace transformers	Executed inside the /summarize route; easily extendable for other offline translation/text tasks.	Medium
3	Unified Tabbed Workspace Layout (index.html)	HTML5 / Jinja2 / Vanilla JS	Act as the shared base dashboard container that dynamically injects views for all 5 feature pipelines.	High

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	4.7	Clean separation into backend, styles, and template files.
Readability	4.8	Follows PEP 8 with clear, intuitive function naming.
Reusability	4	Modular API helpers easily adapt to new features.
Documentation / Comments	4.5	Clear block comments outline model setup and routes.
Overall Score	4.5	Highly optimized hybrid structure balancing local and cloud AI.